

**UNIVERSIDAD GERARDO BARRIOS
FACULTAD DE CIENCIA Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS Y REDES COMPUTACIONALES**



Modulo 4 Desarrollo de aplicaciones con inteligencia artificial

TEMA:

Instrumentación, línea base de rendimiento y plan de escalabilidad

PRESENTADO POR:

**Bryan Orlando Giron Argueta
Gerson Usiel Quintanilla Sánchez
Saúl Emmanuel Zuniga Villatoro**

Docente: Marco Antonio Arévalo Zambrano

EL SALVADOR, SAN MIGUEL, 15 de AGOSTO DE 2026

1. Descripción del servicio y del flujo crítico seleccionado

GameVision IA es el proyecto de graduación del equipo para el Módulo 4, y predice el potencial comercial de videojuegos de Steam mediante un modelo Random Forest entrenado con aproximadamente 58,000 juegos, complementado con un chatbot asistente construido con LangChain y Gemini. El backend está construido en FastAPI (Python 3.11) y desplegado en Render mediante un contenedor Docker de una sola etapa; la base de datos es PostgreSQL administrado por Supabase (con autenticación vía Google OAuth), y el frontend en React/Vite está desplegado en Vercel.

El flujo crítico oficial seleccionado para esta evaluación es POST /api/predict, el endpoint autenticado que realmente utilizan los usuarios de la aplicación en producción: verifica un JWT emitido por Supabase, ejecuta la inferencia del modelo, y persiste tanto la predicción como una nueva sesión de chat asociada. Como comparación auxiliar se instrumentó también POST /api/predict-demo (público, sin autenticación ni escritura en base de datos, pero usando el mismo modelo real), lo que permite aislar el costo del modelo del costo del resto del flujo productivo. Adicionalmente se tomó una muestra pequeña de POST /api/chat, ya que consume cuota externa real de la API de Gemini.

Ambiente de producción evaluado: <https://gamevisionia.onrender.com>

2. Preguntas de observabilidad y campos registrados

Antes de instrumentar, se definieron las preguntas operativas concretas que la observabilidad debía poder responder:

- ¿El servicio responde, y con qué código de estado?
- ¿Cuánto tarda el flujo completo, y cuánto tarda cada etapa interna del modelo?
- ¿Qué está fallando, con qué frecuencia, y de forma controlada (sin exponer datos sensibles)?
- ¿Qué versión del modelo produjo una respuesta específica?
- ¿Cómo se correlaciona una solicitud puntual entre el log, la base de datos y la respuesta HTTP?

Para responderlas, se creó la tabla request_logs en Supabase, con una fila por cada petición HTTP completada en toda la API (no solo en el endpoint de predicción). Los campos registrados son:

Campo	Qué responde	Tipo
request_id	Correlaciona una petición específica entre el log, la fila en BD y la respuesta HTTP (header X-Request-ID)	string
method / path	Qué ruta y verbo se ejecutó	string
status_code	Si la petición terminó bien o con error	int
duration_ms	Cuánto tardó el flujo completo, medido del lado del servidor	float
model_version	Qué versión del modelo respondió (solo en endpoints de predicción)	string
error_type	Clasificación del error (client_error / server_error), sin exponer detalles sensibles	string
validate_ms / feature_prep_ms / inference_ms	Desglose interno de tiempos dentro del modelo, solo para /predict y /predict-demo	float
created_at	Marca de tiempo, usada también para la retención automática	timestamp

Tabla 1. Campos registrados en request_logs.

3. Código de instrumentación

La instrumentación se implementó como un middleware global de FastAPI (main.py), que se ejecuta para cualquier petición que llegue a la API — no está limitado a un único endpoint. Genera un request_id de correlación, mide la duración total con time.perf_counter(), agrega los headers X-Request-ID y X-Process-Time-Ms a la respuesta, registra un evento JSON en stdout (que Render captura automáticamente), y guarda una fila en request_logs como BackgroundTask, es decir, después de haber enviado la respuesta al usuario, para que el registro del log nunca sea, en sí mismo, el cuello de botella:

```
@app.middleware("http")
async def observability_middleware(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    start = time.perf_counter()

    response = await call_next(request)

    duration_ms = round((time.perf_counter() - start) * 1000, 2)
    status_code = response.status_code

    error_type = None
    if status_code >= 500:
        error_type = "server_error"
    elif status_code >= 400:
        error_type = "client_error"

    model_version = getattr(request.state, "model_version", None)
    stage_timings = getattr(request.state, "stage_timings", None) or {}

    response.headers["X-Request-ID"] = request_id
    response.headers["X-Process-Time-Ms"] = f"{duration_ms:.2f}"

    # El insert a Supabase corre en background, después de responder,
    # para no sumarse a la latencia medida ni bloquear al usuario.
    existing_background = response.background
    async def _finalize():
        if existing_background is not None:
            await existing_background()
        _persist_request_log(request_id=request_id, method=request.method,
                             path=request.url.path, status_code=status_code,
                             duration_ms=duration_ms, model_version=model_version,
                             error_type=error_type, **stage_timings)
    response.background = BackgroundTask(_finalize)

    return response
```

Cualquier falla al escribir el log o al insertar en Supabase se atrapa en un try/except dedicado (_persist_request_log) y solo genera un warning — nunca convierte una respuesta exitosa en un error para el usuario.

Evidencia de una solicitud exitosa

Petición real a producción, con los headers de correlación ya presentes en la respuesta:

```
$ curl -i https://gamevisionia.onrender.com/health
```

```
HTTP/1.1 200 OK
x-process-time-ms: 203.51
x-request-id: e63579a8
{"status":"ok","model":"loaded","database":"connected"}
```

Results		Chart			
request_id	method	path	status_code	duration_ms	created_at
c2c93afc	GET	/health	200	204.1	2026-08-14 01:06:01.022848
76d5d5dc	GET	/health	200	204.79	2026-08-14 01:05:56.023941
18754197	GET	/health	200	204.2	2026-08-14 01:05:51.167108
935900f1	GET	/health	200	203.4	2026-08-14 01:05:50.756639
65cc4e62	GET	/health	200	204.09	2026-08-14 01:05:46.023487

Figura 1. Filas reales en request_logs correspondientes a peticiones a /health, con request_id, duración y estado.

4. Evidencia de una entrada inválida o error controlado

Se envió una predicción con un precio negativo (`price_initial = -10`) directamente contra producción, para verificar que un error controlado también queda correlacionado:

```
C:\Users\sauzu>curl -i -X POST https://gameversionia.onrender.com/api/predict-demo -H "Content-Type: application/json" -d '{"price_initial": -10}'
HTTP/1.1 422 Unprocessable Entity
Date: Sat, 15 Aug 2026 02:18:33 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
x-rnd-id: 838866e0cfbf7f-4ab5
Server: cloudflare
vary: Accept-Encoding
x-process-time-ms: 2.51
x-render-origin-server: uvicorn
x-request-id: fed61c2
cf-cache-status: DYNAMIC
CF-RAY: a2b4b51c7f7dde1b-MIA
alt-svc: h3=":443"; ma=86400

{"detail": [{"type": "value_error", "loc": ["body", "price_initial"], "msg": "Value error, El precio no puede ser negativo", "input": -10, "ctx": {"error": {}}], {"type": "missing", "loc": ["body", "is_free"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "release_year"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "release_month"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_indie"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_casual"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_action"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_adventure"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_simulation"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_strategy"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_early_access"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "genre_free_to_play"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_single_player"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_multi_player"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_pvp"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_online_pvp"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_online_coop"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_shared_split_screen"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_shared_split_screen_pvp"], "msg": "Field required", "input": {"price_initial": -10}], {"type": "missing", "loc": ["body", "cat_shared_split_screen_coop"], "msg": "Field required", "input": {"price_initial": -10}}]
```

Figura 2. Respuesta 422 real, con validador de negocio propio ("El precio no puede ser negativo") y headers de observabilidad presentes.

Results Chart				
request_id	status_code	error_type	path	created_at
fed9e1c2	422	client_error	/api/predict-demo	2026-08-15 02:18:33.926502
7791b12e	405	client_error	/	2026-08-14 04:09:04.90222
91e774da	405	client_error	/	2026-08-14 01:02:06.033275

Figura 3. Fila correspondiente en request_logs — el request_id fed9e1c2 coincide exactamente con el del header de la Figura 2, confirmando la correlación end-to-end.

5. Datos excluidos por privacidad o seguridad

El diseño de request_logs registra únicamente metadatos técnicos, nunca contenido de negocio ni credenciales. Explícitamente, nunca se registran:

- Contraseñas, tokens de acceso, ni el encabezado Authorization completo.
- Cadenas de conexión ni credenciales de base de datos.
- El contenido de los mensajes del chat, ni el texto de las respuestas de Gemini.
- El payload completo de una predicción (features del juego) — solo su duración de procesamiento.
- Datos personales del usuario más allá de lo estrictamente necesario para operar (no se guarda ni correo ni nombre en request_logs).

Lo único que persiste es lo listado en la Tabla 1: identificadores de correlación, ruta, estado, duración y clasificación de error — suficiente para diagnosticar el servicio sin exponer información sensible en logs, capturas o en este documento.

6. Escenario y resultados de la medición de rendimiento

La línea base se generó con un script propio (scripts/benchmark_predict.py, incluido en el repositorio), que realiza una petición de calentamiento no contabilizada, seguida de 20 peticiones secuenciales a /api/predict-demo, 20 a /api/predict (con un JWT real de una cuenta de prueba), y una muestra de 3 a /api/chat. El script documenta automáticamente el ambiente, el payload utilizado y el commit de código exacto de cada corrida.

Escenario	p50	p95	máx	n	error
/predict-demo — antes	817 ms	969 ms	1064 ms	20	0%
/predict-demo — después	1021 ms	1195 ms	1295 ms	20	0%
/predict real — antes	1431 ms	1784 ms	2335 ms	20	0%
/predict real — después	1239 ms	1446 ms	1873 ms	20	0%
/chat — antes (muestra 3)	avg 6280 ms	—	12900 ms	3	0%
/chat — después (muestra 3)	avg 12436 ms	—	30761 ms	3	1 timeout

Tabla 2. Resultados client-side (medidos desde la máquina del desarrollador, incluyen red).

Salida completa del script — corrida ANTES de la mejora

```

Despertando el servicio en https://gamevisionia.onrender.com...
  listo en 1310 ms
--- POST /api/predict-demo (sin auth, comparación auxiliar) (20 peticiones secuenciales) ---
[1/20] 200 - 1063.9 ms
[2/20] 200 - 817.8 ms
[3/20] 200 - 821.4 ms
[4/20] 200 - 818.3 ms
[5/20] 200 - 826.2 ms
[6/20] 200 - 914.9 ms
[7/20] 200 - 816.1 ms
[8/20] 200 - 818.1 ms
[9/20] 200 - 819.0 ms
[10/20] 200 - 717.9 ms
[11/20] 200 - 719.6 ms
[12/20] 200 - 908.3 ms
[13/20] 200 - 727.1 ms
[14/20] 200 - 714.0 ms
[15/20] 200 - 963.6 ms
[16/20] 200 - 572.1 ms
[17/20] 200 - 609.4 ms
[18/20] 200 - 727.6 ms
[19/20] 200 - 709.4 ms
[20/20] 200 - 730.6 ms
p50=816.96ms p95=968.62ms max=1063.91ms error_rate=0.0%
--- POST /api/predict (real, flujo crítico de producción) (20 peticiones secuenciales) ---
[1/20] 200 - 2335.5 ms
[2/20] 200 - 1535.3 ms
[3/20] 200 - 1409.8 ms
[4/20] 200 - 1308.3 ms
[5/20] 200 - 1316.6 ms
[6/20] 200 - 1396.4 ms
[7/20] 200 - 1326.1 ms
[8/20] 200 - 1431.5 ms
[9/20] 200 - 1551.7 ms
[10/20] 200 - 1417.4 ms
[11/20] 200 - 1755.0 ms
[12/20] 200 - 1519.6 ms
[13/20] 200 - 1436.5 ms
[14/20] 200 - 1427.7 ms
[15/20] 200 - 1430.9 ms
[16/20] 200 - 1435.4 ms
[17/20] 200 - 1431.2 ms
[18/20] 200 - 1545.6 ms
[19/20] 200 - 1421.7 ms
[20/20] 200 - 1536.2 ms
p50=1431.37ms p95=1784.03ms max=2335.46ms error_rate=0.0%
--- POST /api/chat (muestra de 3, session_id=71) ---
[1/3] 200 - 12900.0 ms
[2/3] 200 - 2558.1 ms
[3/3] 200 - 3383.2 ms

=== RESUMEN ===
- POST /api/predict-demo: p50=816.96ms p95=968.62ms max=1063.91ms error_rate=0.0%
- POST /api/predict (real): p50=1431.37ms p95=1784.03ms max=2335.46ms error_rate=0.0%
- POST /api/chat (muestra chica): avg=6280.44ms max=12900.05ms

```

Salida completa del script — corrida DESPUÉS de la mejora

```

Despertando el servicio en https://gamevisionia.onrender.com...
  listo en 53471 ms  <- cold-start real capturado (servicio dormido)

```

```

--- POST /api/predict-demo (sin auth, comparación auxiliar) (20 peticiones secuenciales) ---
[1/20] 200 - 1189.9 ms
[2/20] 200 - 1295.3 ms
[3/20] 200 - 958.4 ms
[4/20] 200 - 1126.9 ms
[5/20] 200 - 1021.8 ms
[6/20] 200 - 921.1 ms
[7/20] 200 - 1022.7 ms
[8/20] 200 - 923.4 ms
[9/20] 200 - 1124.9 ms
[10/20] 200 - 922.8 ms
[11/20] 200 - 1027.5 ms
[12/20] 200 - 822.5 ms
[13/20] 200 - 884.0 ms
[14/20] 200 - 1024.8 ms
[15/20] 200 - 1022.9 ms
[16/20] 200 - 824.6 ms
[17/20] 200 - 1020.7 ms
[18/20] 200 - 919.4 ms
[19/20] 200 - 1129.5 ms
[20/20] 200 - 915.6 ms
p50=1021.26ms p95=1195.21ms max=1295.32ms error_rate=0.0%
--- POST /api/predict (real, flujo crítico de producción) (20 peticiones secuenciales) ---
[1/20] 200 - 1872.7 ms
[2/20] 200 - 1198.3 ms
[3/20] 200 - 1332.0 ms
[4/20] 200 - 1263.5 ms
[5/20] 200 - 1296.3 ms
[6/20] 200 - 1224.6 ms
[7/20] 200 - 1237.4 ms
[8/20] 200 - 1423.9 ms
[9/20] 200 - 1229.3 ms
[10/20] 200 - 1228.1 ms
[11/20] 200 - 1227.9 ms
[12/20] 200 - 1334.3 ms
[13/20] 200 - 1226.1 ms
[14/20] 200 - 1285.1 ms
[15/20] 200 - 1321.7 ms
[16/20] 200 - 1126.7 ms
[17/20] 200 - 1240.2 ms
[18/20] 200 - 1318.2 ms
[19/20] 200 - 1224.8 ms
[20/20] 200 - 1127.0 ms
p50=1238.83ms p95=1446.33ms max=1872.71ms error_rate=0.0%
--- POST /api/chat (muestra de 3, session_id=91) ---
[1/3] 200 - 3275.6 ms
[2/3] 200 - 3271.1 ms
[3/3] ERROR (ReadTimeout) - 30760.6 ms    <- ver Figura 6 / Sección 7

=== RESUMEN ===
- POST /api/predict-demo: p50=1021.26ms p95=1195.21ms max=1295.32ms error_rate=0.0%
- POST /api/predict (real): p50=1238.83ms p95=1446.33ms max=1872.71ms error_rate=0.0%
- POST /api/chat (muestra chica): avg=12435.79ms max=30760.63ms

```

Comparar directamente el p50 de /predict entre las dos corridas es engañoso, porque el "control" (/predict-demo, que no se modificó) también varió entre corridas por condiciones de red del día. La comparación confiable es la interna, medida del lado del servidor mediante el desglose de etapas ya guardado en request_logs:



Figura 4. Desglose interno ANTES de la mejora — duración total promedio 781.06 ms.

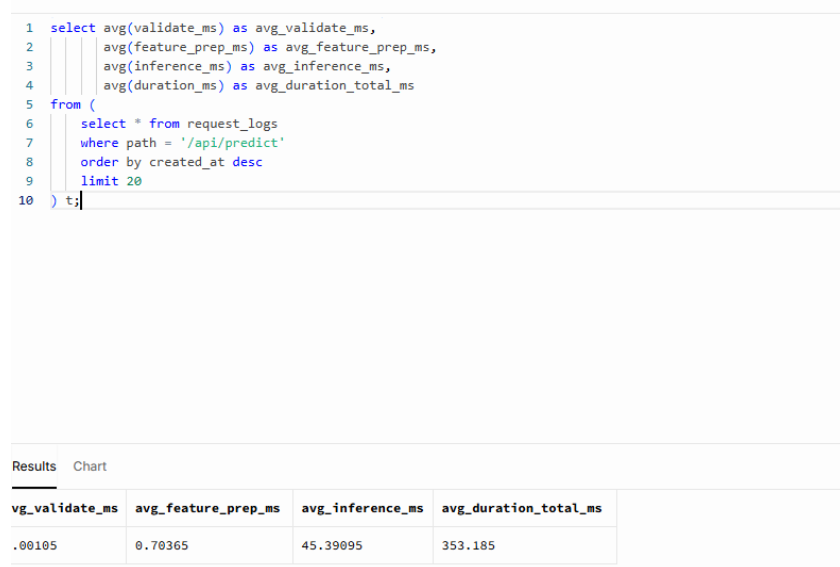


Figura 5. Desglose interno DESPUÉS de la mejora — duración total promedio 353.19 ms.

Componente (servidor)	Antes	Después	Cambio
Duración total (duration_ms)	781.06 ms	353.19 ms	-54.8%
Overhead no-modelo (auth + BD)	741.53 ms	307.09 ms	-58.6%
Modelo (validate+feature_prep+inference)	39.53 ms	46.10 ms	sin cambio significativo

Tabla 3. Desglose interno antes/después, medido del lado del servidor (sin ruido de red).

7. Cuello de botella, riesgo o comportamiento inesperado identificado

El modelo de IA no es el cuello de botella

La Tabla 3 muestra que el modelo Random Forest representa entre el 3% y el 13% del tiempo total de una predicción real — el resto es autenticación JWT y escritura en Supabase. Optimizar el modelo no tendría impacto perceptible en la experiencia del usuario; el cuello de botella está en la infraestructura alrededor del modelo, no en la inteligencia artificial en sí.

Cold-start del plan gratuito de Render

En una de las corridas del script, el servicio llevaba más de 15 minutos sin tráfico y se confirmó el cold-start documentado desde Semana 4 con un número real: la primera petición de calentamiento tardó 53,471 ms (53.5 segundos) antes de que el servicio respondiera. Es la restricción de disponibilidad más severa identificada en todo el proyecto.

Comportamiento inesperado: latencia extrema sin timeout en el chat

Durante la segunda corrida del benchmark, la tercera llamada a `/api/chat` no respondió dentro del timeout de 30 segundos configurado en el script de pruebas (`ReadTimeout`). Sin embargo, la fila correspondiente en `request_logs` muestra que el servidor sí completó la petición — solo que tardó 177,173.93 ms (más de 2 minutos y 55 segundos) con `status_code` 200:

Results		Chart		
request_id	status_code	duration_ms	error_type	created_at
bd1d80f6	200	177173.93	NULL	2026-08-14 04:32:35.875944
2c738503	200	2371.08	NULL	2026-08-14 04:29:37.759274
1de1e4bc	200	2325.78	NULL	2026-08-14 04:29:34.44522
79227c93	200	2427.31	NULL	2026-08-14 03:41:01.375929
c03c821c	200	1838.31	NULL	2026-08-14 03:40:58.00064

Figura 6. Fila real en `request_logs`: la petición se completó (200), pero tardó 177 segundos — muy por encima de lo que un usuario esperaría.

El backend actualmente no define ningún timeout explícito en la llamada de LangChain hacia Gemini, por lo que si la API de Google responde lento, el servidor espera indefinidamente. Un usuario real probablemente recargaría la página mucho antes de esos 177 segundos, sin saber que su mensaje sí se estaba procesando.

8. Mejora aplicada y comparación antes/después

Mejora aplicada: reducir viajes de red a Supabase en `/predict`

El código original de `POST /api/predict` ejecutaba dos ciclos independientes de `commit()` + `refresh()` — uno para la predicción y otro para la sesión de chat — sumando 4 viajes de red secuenciales a la base de datos por petición. Se reemplazó por `flush()` (que obtiene el `id` y `created_at` generados por PostgreSQL vía `RETURNING`, en el mismo viaje del `INSERT`) y un único `commit()` final:

Antes

```

db.add(db_prediction)
db.commit()
db.refresh(db_prediction)

chat_session = ChatSession(prediction_id=db_prediction.id)
db.add(chat_session)
db.commit()
db.refresh(chat_session)

```

Después

```

db.add(db_prediction)
db.flush() # INSERT ... RETURNING id, created_at - 1 viaje
prediction_id = db_prediction.id
prediction_created_at = db_prediction.created_at

chat_session = ChatSession(prediction_id=prediction_id)
db.add(chat_session)
db.flush()
session_id = chat_session.id # leído ANTES del commit

db.commit()

```

Detalle relevante: SessionLocal usa `expire_on_commit=True` (valor por defecto de SQLAlchemy), por lo que leer un atributo del objeto después de `db.commit()` dispara automáticamente un `SELECT` adicional para "refrescarlo". Por eso `session_id` y `prediction_created_at` se leen en variables de Python antes del commit final, evitando reintroducir el viaje de red que se buscaba eliminar.

El resultado (Tabla 3) confirma que la mejora fue quirúrgica: el overhead de auth+BD cayó 58.6%, mientras que el tiempo del modelo se mantuvo estadísticamente igual — evidencia de que el cambio afectó únicamente lo que se pretendía optimizar.

Mejora propuesta (no implementada): timeout explícito en la llamada a Gemini

A partir del hallazgo de la Figura 6, se propone agregar un timeout explícito (por ejemplo 15-20 segundos) a la llamada de LangChain hacia la API de Gemini, junto con un mensaje de error controlado hacia el usuario si se excede. Esto no elimina la latencia ocasional de la API externa, pero convierte una espera indefinida en un fallo predecible y comunicado, mejorando la experiencia del usuario sin requerir cambios de infraestructura.

9. Plan de escalabilidad, limitaciones pendientes y repositorio

¿Qué crecimiento se está considerando y cuál es la restricción observada?

GameVision IA es un proyecto académico con tráfico bajo y esporádico; no se espera crecimiento masivo de usuarios concurrentes en el corto plazo. La restricción real observada no es de cómputo, sino de disponibilidad: el cold-start del plan gratuito de Render (53.5 segundos tras 15 minutos de inactividad) y la latencia de red base entre El Salvador y la región del servidor.

¿Qué mejora debe realizarse primero?

Según el diagnóstico de la Tabla 3, el orden de prioridad basado en evidencia es: (1) ya aplicada — reducir los viajes de red a Supabase en /predict; (2) agregar el timeout explícito a la llamada de Gemini, dado el hallazgo de 177 segundos sin control; (3) evaluar si eliminar el cold-start (plan pago de Render, o un servicio externo de ping periódico) se justifica según qué tan crítica sea la disponibilidad instantánea para la evaluación y demostración del proyecto. Optimizar el modelo de Random Forest no es prioritario: ya demostró ser liviano (39-46 ms).

¿Cuándo convendría usar caché, workers, colas o más instancias?

- Caché: las llaves JWKS de Supabase ya se cachean 5 minutos vía PyJWKClient; no se identificó otro caso claro de caché, ya que cada predicción depende de un payload distinto.
- Colas / procesamiento asíncrono: el chat es el candidato más claro, dado el outlier de 177 segundos — si ese patrón se repite de forma consistente (no como caso aislado), desacoplar la respuesta del chat de una espera HTTP síncrona sería la siguiente mejora estructural.
- Más workers o instancias: solo se justificaría ante tráfico concurrente real y sostenido. Con los datos actuales (proyecto académico, sin usuarios concurrentes reales) no está justificado, y cada worker adicional duplicaría el uso de memoria del modelo (63 MB por proceso).

¿Qué impacto tendría en memoria, datos, privacidad y costo?

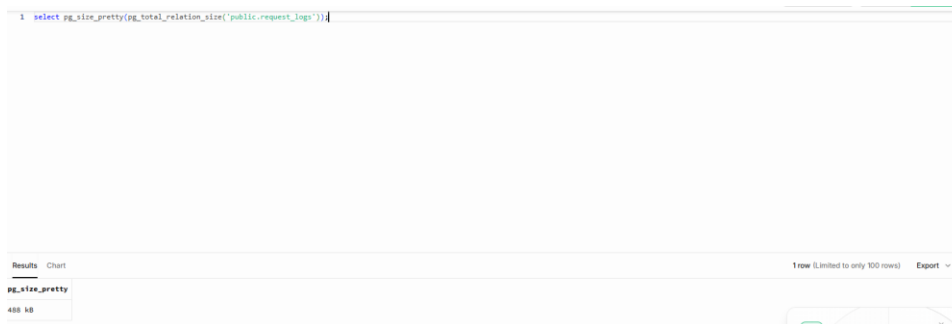


Figura 7. Tamaño actual de request_logs en Supabase: 488 kB.

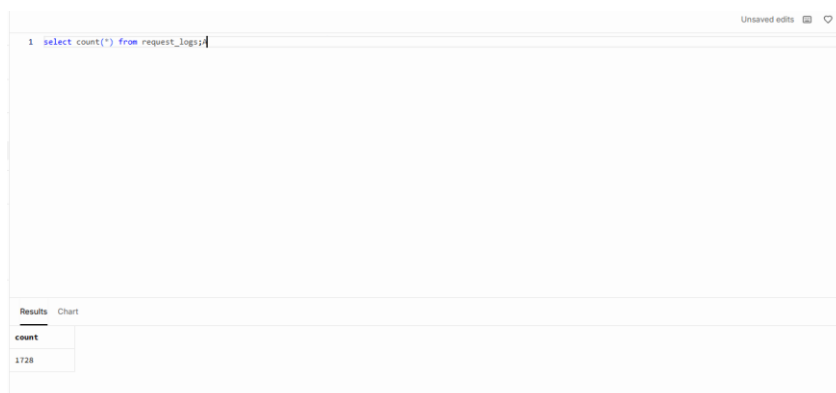


Figura 8. Cantidad de filas correspondiente a ese tamaño: 1,728.

Con 488 kB para 1,728 filas (≈ 0.28 KB por fila), y un límite de 500 MB en el plan gratuito de Supabase, se necesitarían aproximadamente 1.81 millones de filas para agotar el espacio. Esas 1,728 filas se generaron en poco más de un día de tráfico anormalmente alto (dos corridas

completas del benchmark, más pruebas manuales); al ese ritmo sostenido tomaría cerca de 1,048 días (≈ 3 años) llegar al límite — y en uso normal, sin sesiones intensivas de pruebas, sería considerablemente más lento. Más importante aún: gracias al job de retención (pg_cron) ya configurado, la tabla nunca crece sin control, porque cada fila se elimina automáticamente a los 30 días — la restricción de almacenamiento fue diseñada para dejar de ser un problema, no solo estimada como poco probable.

En privacidad, escalar no cambia el diseño: request_logs nunca almacenó tokens, contraseñas ni contenido de mensajes (sección 5), por lo que agregar más tráfico no incrementa el riesgo de exposición de datos sensibles. En costo, el plan gratuito de Render no permite más de una instancia; escalar horizontalmente implicaría pasar a un plan pago (aproximadamente desde 7 USD/mes) tanto para eliminar el cold-start como para soportar múltiples workers.

¿Qué indicador permitiría decidir cuándo escalar?

- p95 de duration_ms en request_logs sostenido por encima de un umbral (por ejemplo, 2 segundos) durante varios días consecutivos, no solo en una corrida aislada.
- error_rate sostenido por encima de 1-2% en cualquier endpoint, medido con la misma tabla.
- Proyección del crecimiento de filas en request_logs indicando que el límite de almacenamiento se alcanzaría en menos de 90 días, pese a la retención activa.
- Frecuencia de timeouts o respuestas superiores a 15 segundos en /api/chat, como el identificado en la Figura 6.

Limitaciones pendientes

- Las 20 peticiones de cada escenario se ejecutaron de forma secuencial, no concurrente — no se midió el comportamiento bajo múltiples usuarios simultáneos.
- El tamaño de muestra ($n=20$, y $n=3$ para el chat) es suficiente para el mínimo que exige esta evaluación, pero estadísticamente limitado para un SLA formal.
- El timeout explícito para la llamada a Gemini quedó como mejora propuesta, no implementada.
- El cold-start solo se capturó una vez de forma oportunista, no como experimento controlado repetible.

Enlace al repositorio

Repositorio: <https://github.com/sauzuniga/GameVisionIA>

Backend en producción: <https://gamevisionia.onrender.com>

Frontend en producción: <https://game-vision-ia.vercel.app>

El repositorio incluye el middleware de observabilidad (backend/main.py), el modelo de la tabla request_logs (backend/models.py), el script de línea base (backend/scripts/benchmark_predict.py), el script de retención (backend/scripts/setup_retention.sql), y los resultados de ambas corridas en formato JSON (backend/docs/evidencias/).